



Передовые
инженерные
школы



МИЭТ

НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ

Передовая инженерная школа «Средства проектирования и
производства электронной компонентной базы»

Магистерская программа «Вычислительные системы и ЭКБ»

Дисциплина:

«Маршрут проектирования цифровых интегральных схем»

Лабораторная работа 01 – RTL-разработка и функциональная верификация

Кузьмин Павел Андреевич

2026 год



Цель курса — пройти основные этапы создания цифровой микросхемы: от разработки и проверки RTL-описания на SystemVerilog до получения файлов физической реализации, которые используются для передачи проекта на фабрику для производства микросхемы.

Программа курса

ЛР №1 — RTL-разработка и функциональная верификация

Разработка RTL на SystemVerilog, создание testbench и функциональное моделирование.

ЛР №2 — Логический синтез и формальная проверка

Синтез RTL в библиотечные элементы, анализ результатов синтеза и проверка эквивалентности RTL и netlist.

ЛР №3 — Floorplanning и подготовка физической реализации

Создание floorplan, размещение выводов и подготовка дизайна к размещению стандартных ячеек.

ЛР №4 — Placement, CTS и Routing

Размещение стандартных ячеек, построение тактового дерева и трассировка соединений.

ЛР №5 — Анализ падения напряжения (IR Drop)

Анализ сети питания физически реализованного проекта, оценка падения напряжения и выявление проблемных участков.

ЛР №6 — Post-layout моделирование

Gate-level моделирование физически реализованного проекта с учётом задержек.

ЛР №7 — Интеграция системы верхнего уровня

Объединение разработанных функциональных блоков в модуль верхнего уровня, реализация межблочных соединений и логики TOP, функциональная проверка интегрированной системы.

Оценка определяется по **качеству выполненной работы и уровню понимания материала на защите.**

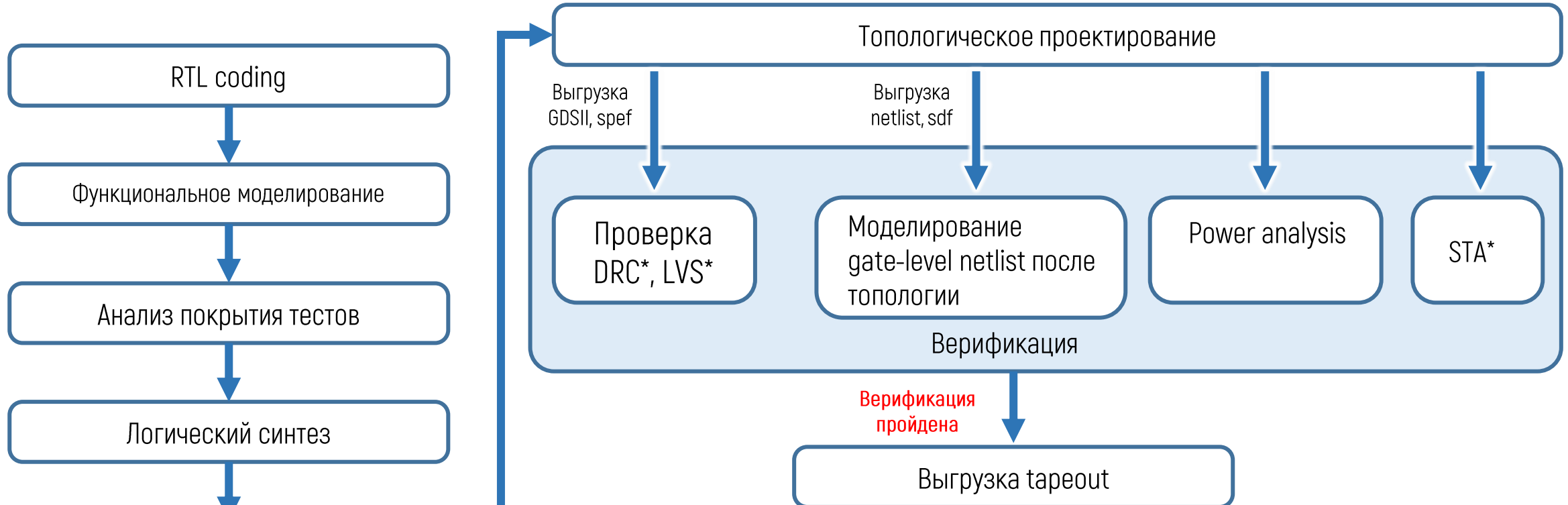
Оценка	Требования
5 — отлично	Работа выполнена полностью. Студент уверенно объясняет свою реализацию, полученные результаты и понимает место рассматриваемого этапа в маршруте проектирования.
4 — хорошо	Работа выполнена, основные требования соблюдены. Студент понимает основные принципы, но допускает отдельные ошибки или затрудняется в некоторых вопросах.
3 — удовлетворительно	Работа выполнена в минимально достаточном объёме. Студент понимает основные действия и результаты, но имеет существенные пробелы в понимании деталей.

Возможность повысить оценку

Срок защиты	Возможность исправления	Максимальная оценка
В течение 4 лаб после выдачи	Можно исправлять замечания и повторно защищать работу	5
Более 4 лаб после выдачи и до окончания курса	Работу можно исправить и повторно защитить	4
После окончания курса	Лабораторную работу можно сдать, но без возможности получить более высокую оценку	3



Маршрут RTL-to-GDS



DFT (design for testability) - синтез с определением scan цепочек, которые используются для проверки работоспособности тактируемых элементов.

DRC (Design Rule Check) – проверка правилам и стандартам, установленными фабрикой

LVS (Layout Versus Schematic) – проверка соответствия между schematic и la

STA (Static Timing Analysis)

Tapeout – файлы готовые к отправке на фабрику



От выбранного FDK (foundry design kit) зависит **время и стоимость** разработки топологии СФ-блока.

Технологический процесс 28нм

Классификация библиотек входящих в FDK:

- Количество трексов: 7t/9t/12t;
- Пороговое напряжение: ulvt/lvt/svt/hvt/uhvt
- Длина канала: 30n/35n/40n;



30ulvt	30lvt	30svt	30hvt	30uhvt
35ulvt	35lvt	35svt	35hvt	35uhvt
40ulvt	40lvt	40svt	40hvt	40uhvt

Рис.3 Список библиотек FDK без учета количества трексов

7 трексов (High Density) – низкочастотные микропроцессорные карты, SIMcard

9 трексов – стандартное количество;

12 трексов (High Speed) – высокочастотные процессоры интерфейсы: DDR, PCIe и т.д.

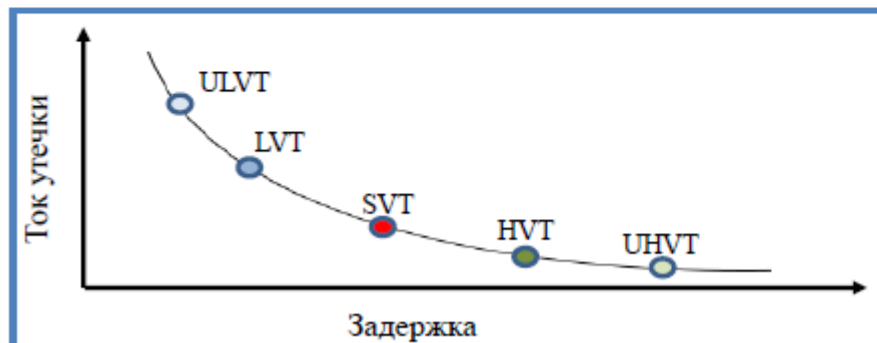


Рис.2 Задержки и токи утечки при разном пороговом напряжении

В зависимости от состава FDK предоставляемого фабрикой. Стоимость может отличаться:

- Доступны все 45 библиотек;
- Доступна только опция 35SVT (Остальные доплата)

Дополнительные производственные расходы:

- Маски (HVT, LVT): 40 тыс. \$
- К стоимости пластины за HVV, LVT: 200 \$

Характеризованная библиотека стандартных элементов:

- Библиотека: 6 тыс. \$



Порядок запуска

Структура папок

```
▼ digital_flow_2026
  ▼ lab1
    ▼ data\rtl\counter
      ≡ counter_tb.sv
      ≡ counter.sv
    > doc
  ▼ run
  ▼ scripts
    $ run.sh
  M Makefile
  ⓘ README.md
  > PDK
```

1. Заходим в папку lab1
2. В терминале пишем
module load cadence/XCELIUMMAIN/24.03.001
3. Запускаем
 1. make sim – для моделирования с gui
 2. make sim_nogui – для моделирования без gui

В качестве примера доступен RTL блока counter и его tb.
Для использования своего блока необходимо внести правки в Makefile

```
SIM_TOP = counter_tb

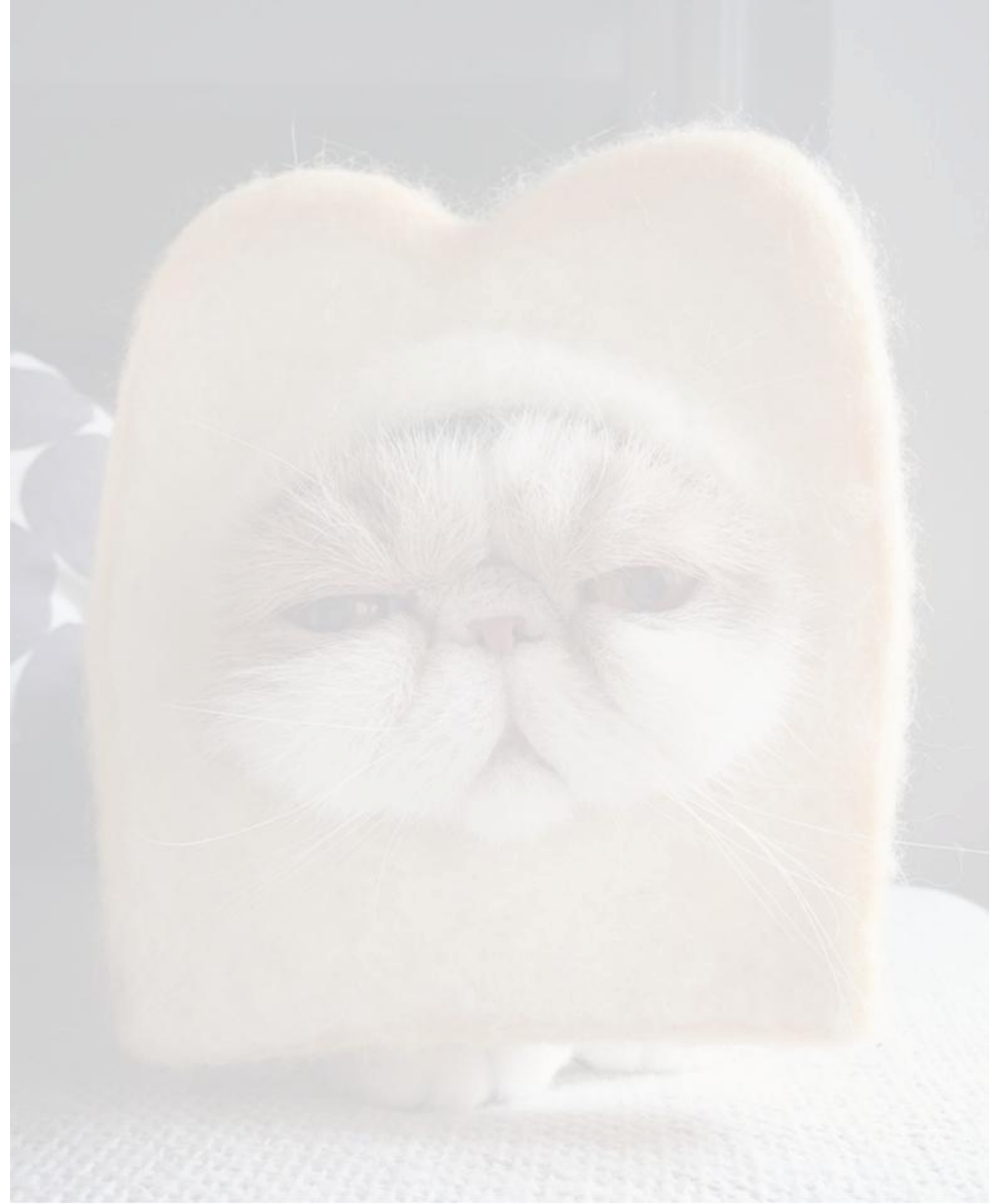
SIM_RTL_DIR = $(ENV_DESIGN_DIR)/rtl/counter

SIM_FILES = \
  $(SIM_RTL_DIR)/counter.sv \
  $(SIM_RTL_DIR)/counter_tb.sv
```

Лабораторное задание

Подробное описание задания, требования к разрабатываемым модулям и их функциональной верификации приведены в файле **README** в папке лабораторной работы.

Далее рассмотрим **общую идею лабораторной работы**, структуру разрабатываемой системы и основные этапы её выполнения.





Подробное описание задания и все требования к лабораторной работе приведены в файле README.md в папке лабораторной работы.

В рамках лабораторной работы необходимо:

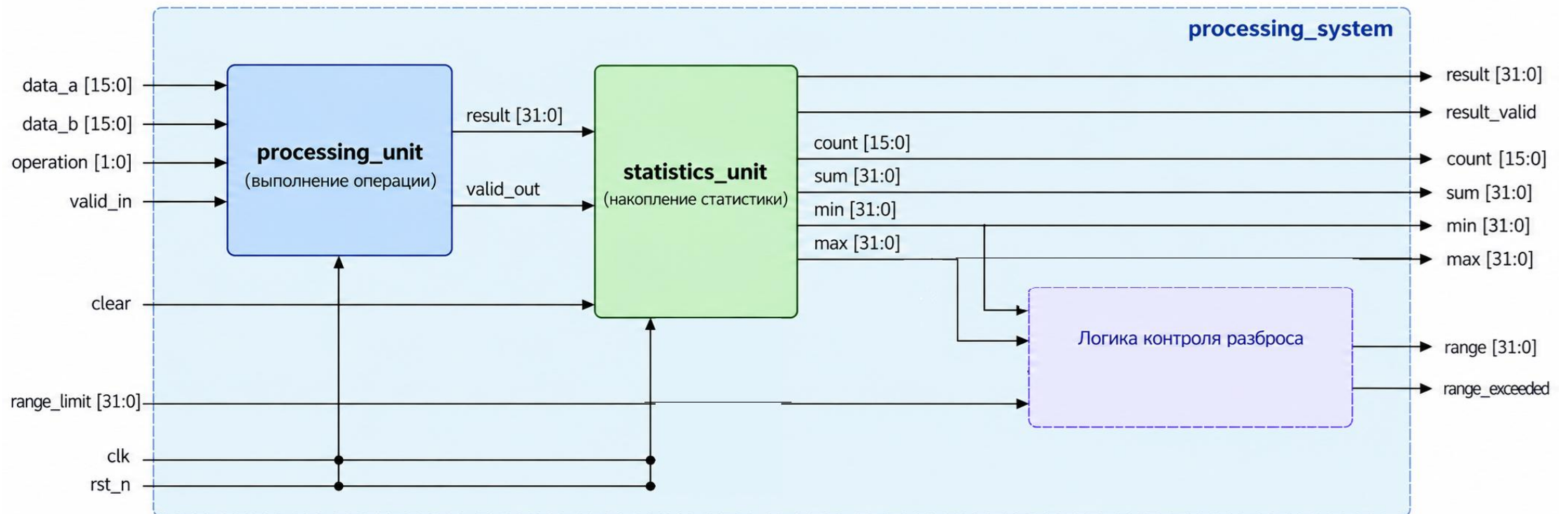
- разработать RTL-описание цифровой системы на языке **SystemVerilog**;
- разработать **testbench** и провести функциональную верификацию отдельных блоков;
- объединить разработанные блоки в **модуле верхнего уровня (TOP)**;
- проверить совместную работу системы с помощью моделирования.



Структура разрабатываемой системы

В лабораторной работе необходимо разработать систему **processing_system**, состоящую из двух функциональных блоков.

- **processing_unit** - Выполняет арифметические и логические операции над входными данными:
- **statistics_unit** - Получает результаты вычислений и накапливает статистическую информацию
- **processing_system** - Объединяет разработанные блоки и реализует дополнительную логику верхнего уровня:





Лабораторная работа выполняется командами по два человека. Допускается индивидуальное выполнение работы.

При работе в команде функциональные блоки распределяются между участниками:

- один студент разрабатывает RTL-описание и testbench для `processing_unit`;
- второй студент разрабатывает RTL-описание и testbench для `statistics_unit`.

После проверки отдельных блоков участники совместно выполняют интеграцию системы: разрабатывают модуль верхнего уровня `processing_system` и testbench `processing_system_tb`.

Каждый студент должен понимать интерфейсы обоих функциональных блоков, их взаимодействие в составе `processing_system` и общую работу системы. При этом на защите студент должен уметь подробно объяснить RTL-описание и верификацию блока, за разработку которого он отвечал.



1. Ознакомиться с заданием и интерфейсами разрабатываемых модулей.
2. При выполнении работы в команде распределить блоки ``processing_unit`` и ``statistics_unit`` между участниками.
3. Разработать RTL-описание своего функционального блока в соответствии с заданным интерфейсом и требованиями к его поведению.
4. Разработать testbench для функциональной проверки блока. Провести моделирование и убедиться в корректности его работы.
5. Объединить разработанные блоки в модуле верхнего уровня ``processing_system`` и реализовать предусмотренную заданием логику верхнего уровня.
6. Разработать ``processing_system_tb`` и проверить совместную работу блоков в составе системы.
7. Исправить обнаруженные ошибки и повторить моделирование до успешного прохождения всех предусмотренных проверок.
8. Подготовить проект и результаты моделирования к защите лабораторной работы.



Для защиты лабораторной работы студент может выбрать один из двух форматов (см. следующий слайд):

1. защита с предварительно подготовленным отчётом;
2. полная очная защита без отчёта.

Независимо от выбранного формата требования к выполненной работе и уровню понимания разработанного проекта остаются одинаковыми.

Устная часть защиты

Независимо от выбранного формата защита завершается кратким обсуждением работы с преподавателем.

Студент должен понимать назначение и работу разработанной системы, уметь объяснить RTL-описание и testbench своего функционального блока, работу модуля ``processing_system``, а также интерпретировать результаты функционального моделирования.

При командном выполнении каждый студент должен подробно понимать разработанный им функциональный блок и соответствующий testbench, а также понимать назначение, интерфейс и основное поведение блока второго участника.



Защита с отчётом

При выборе данного формата студент заранее подготавливает краткий технический отчёт в doc, в котором должны быть представлены:

- описание разработанной системы и её структуры;
- описание реализации функциональных блоков и модуля верхнего уровня;
- описание проведённых проверок;
- результаты функционального моделирования;
- временные диаграммы, подтверждающие корректность работы устройства;
- краткие выводы по результатам работы;

Полные RTL-описаний и testbench включать в отчёт не требуется. Исходный код проекта должен быть доступен преподавателю отдельно.

При наличии отчёта подробная демонстрация всех этапов работы во время защиты не требуется, если соответствующие результаты представлены в отчёте и не вызывают дополнительных вопросов.

Очная защита без отчёта

При выборе данного формата студент должен непосредственно на защите продемонстрировать выполненную работу и результаты функциональной верификации.

В ходе демонстрации необходимо показать и объяснить те же результаты, которые требуются для отчёта: структуру разработанной системы, особенности реализации, проведённые проверки, результаты моделирования и временные диаграммы.